UNIVERSITY OF WOLLONGONG DUBAI
MAAI901 - DEEP LEARNING FOR VISUAL DATA
DR. MILAN DORDEVIC

Fine-Tuning DETR for Maritime Object Detection

Reece Bridle (8233718)

June 3, 2026

CONTENTS

1 Executive Summary

This project develops a deep learning system for maritime object detection using DETR, a DEtection TRansformer. The goal is to identify maritime objects in images, including ships, boats, buoys, and other related objects, using bounding boxes.

The system is developed using a maritime image dataset with accompanying COCO annotations, it includes image files, object categories, and bounding box labels. The pipeline consists of data loading, annotation parsing, data preprocessing, model fine-tuning, and model evaluation and comparison. A pretrained DETR model is used to apply fine-tuning and transfer learning, this allows the model to be optimized for the maritime dataset.

Three training configurations were tested, in three different experiments. The model that performed best used a learning rate of 1×10^{-5} for 10 epochs. It achieved a validation loss of 0.613, mAP@50 of 0.544, and a mean recall of 0.463. This shows that fine-tuning improves DETR's ability to detect maritime objects, while showcasing challenges such as small objects, overlapping vessels, poor visibility, class imbalance, and generalization in complex environments.

2 Introduction and Problem Definition

Maritime images are complicated, because they often include many objects, this can include boats, ships, buoys, sail boats, and other items. Identifying these items plays a vital role in real-world applications such as unmanned surface vehicles (USVs), maritime surveillance, port monitoring, and collision avoidance. In many cases, being dependent on manual oversight is too slow or impractical, particularly in autonomous systems.

The objective of this project is to detect and localise specific objects within maritime images. Contrary to standard image classification, where one label is assigned to an image by a model, object detection requires multiple entities to be detected and located using a bounding box. This can be difficult in maritime imagery because objects may be small, overlapping, far from the camera, or affected by lighting and weather.

This project uses DETR, a Transformer-based object detection model, to detect maritime objects from a dataset annotated with COCO annotations. DETR was picked for this use case because it uses convolutional feature extraction with transformer attention mechanisms, allowing the model to consider relationships between objects across the full image (Carion et al. 2020). The pipeline contains dataset preparation, preprocessing, model fine-tuning, evaluation, and analysis of detection results.

The goal is to build and evaluate a real-world maritime object detection system that shows how deep learning can be applied to a realistic visual problem. This correlates with the assignment focus on advanced computer vision models, fine-tuning, and evaluation.

3 Dataset Selection and Justification

Figure 3.1: Example image from the Singapore Maritime Dataset showing ground-truth object annotations and bounding boxes.

The Singapore Maritime Dataset was selected for this project (Maritime 2023). It was obtained from Roboflow Universe and contains images with annotations in Common Objects in Context (COCO) format for object detection tasks (Lin et al. 2015). An example image from the dataset with the original annotations is shown in Figure 3.1. The dataset is suitable because it matches the project goal of detecting maritime objects such as ships, boats and other objects within an image.

The COCO annotation format means that each image has multiple bounding box coordinates and an object label for each box. This is required for object detection, as the model needs to learn both what an object is and where it is located within the image. The COCO format is compatible with DETR, this allows seamless processing of the images, labels and bounding boxes.

The dataset was also chosen due to its realism, and practicality for fine-tuning. It has 5446 usable images, and 41747 annotations across 10 object classes. This means there is enough for training, validation, and testing splits. In our dataset, each annotation represents one labelled object instance with a corresponding bounding box. Therefore, there are more annotations than images because the majority of the images contain multiple objects.

Table 3.1 Summarises the main dataset characteristics.

The label distribution was also inspected to understand the distribution of the object classes. As shown in Figure 3.2, the Vessel-ship class appears significantly more than other classes, showing a clear class imbalance. This is a shortcoming, as the model will learn the common classes more effectively, while rarer classes like Flying bird-plane will be harder to reliably detect.

Property	Value
Dataset Type	Maritime Imagery
Source	Roboflow Universe
Annotation Format	COCO JSON
Number of Images	5446
Number of Annotations	41747
Number of Classes	10
Training Images	4356 (80%)
Validation Images	545 (10%)
Testing Images	545 (10%)

Table 3.1: Main dataset characteristics.

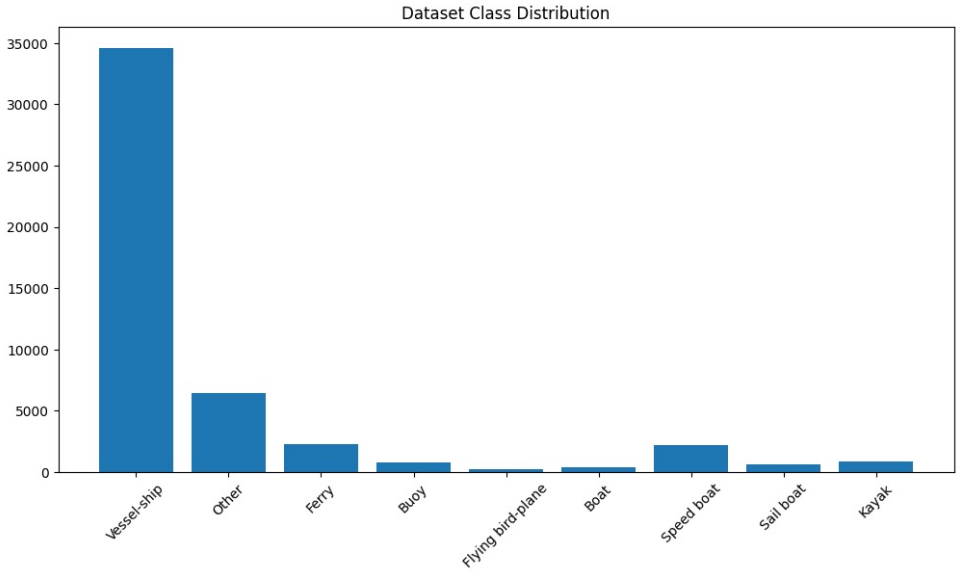

Figure 3.2: Bar chart of the distribution of the dataset classes.

On the whole, the dataset was chosen for this system because it provides real maritime imagery with structured COCO annotations, ten object categories, and sufficient examples for fine-tuning and evaluation of the pretrained DETR model.

4 Preprocessing Strategy

4.1 Dataset Validation and Cleaning

A validation process was performed to identify missing or invalid image files referenced within the annotation file. Any images which did not have corresponding annotations were removed from the dataset, and any annotations which did not link to a valid image were removed. This cleaning step ensured that all remaining annotations referred to valid images and prevented errors during training and evaluation.

4.2 Dataset Exploration and Verification

After cleaning, we spent time looking at the data itself. Firstly, we checked the object types and how the classes were spread out. This allowed us to check for any obvious imbalance. Next, we loaded images along with their ground-truth bounding boxes to ensure annotations loaded without issues and the parsing step worked as intended.

4.3 Dataset Splitting

Splitting came next. The data was divided into training, validation, and testing parts using an 80/10/10 split. This left 4356 images for training along with 545 for both the validation and testing sets. The training set was used to fine-tune the model, while the validation set was used to track model progress during training. Importantly, the testing set was only used for final evaluation and was not used for training or selection. This strategy allows us to evaluate a model's ability to predict on unseen data, without bias.

4.4 DETR Input Preparation

To prepare the dataset for fine-tuning, label mappings were created from the COCO annotation file. This included two mapping dictionaries, id2label which converts category numbers into class names and a matching label2id dictionary, which converts human-readable category names into numbers for the model.

Input preparation specific to DETR was handled using the Hugging Face DetrImageProcessor. In the custom dataset class which was created, each image and its annotations were passed to the processor. The processor converted them into the format expected by DETR. This includes resizing, normalising the image, converting it into tensors, and formatting the bounding boxes and class labels for training.

The resulting pipeline produces a clean, verified, and correctly formatted dataset, which is ready for fine-tuning a DETR model on the maritime object detection task.

5 Model Selection and Configuration

5.1 DETR Architecture Selection

The DEtection TRansformer (DETR) architecture was chosen for this project due to its strong performance on object detection tasks and its ability to detect multiple objects within a single image. Unlike traditional detection approaches that rely on anchor boxes and non-maximum suppression, DETR carries out direct set prediction using a Transformer architecture (Carion et al. 2020). This allows the model to learn object localisation and classification while still considering the relationships between objects in the image.

Furthermore, DETR is appropriate for maritime prediction as the images often contain multiple vessels of different sizes, distances, and positions. Additionally, objects can be occluded, closely grouped, or far from the camera. The self-attention mechanism used by transform-

ers allows the model to obtain global contextual information, helping it identify objects within complex maritime imagery.

To decrease training time and optimize performance, transfer learning was applied using a pretrained DETR model. The implementation used the Facebook DETR ResNet-50 model (facebook/detr-resnet-50) available on the Hugging Face Transformers library (Carion et al. 2020). This model was originally pretrained on the COCO dataset and therefore already had strong feature extraction capabilities for object detection tasks.

DETR uses the ResNet-50 convolutional neural network to extract visual features from input images before they are processed by the Transformer encoder-decoder architecture. This combination allows the model to capture local image features and global feature relationships, making it suitable for maritime object detection.

5.2 Model Configuration

The pretrained DETR model was adapted to the maritime dataset by replacing the original classification layer. Instead of using its original categories, it uses the categories provided in the Singapore Maritime Dataset. Since the original DETR model was pretrained on the COCO dataset, which contains 91 categories (Lin et al. 2015), the classification layer was recreated and matches the maritime dataset classes. This allowed the pretrained feature extraction layers to be retained while adapting the model to the new task.

Custom id2label and label2id mappings were generated from the COCO annotation file to ensure consistency between the dataset labels and model predictions. The final model was configured to detect the following object categories:

- Boat
- Buoy
- Ferry
- Flying bird-plane (Bird or Plane)
- Kayak
- Sail Boat
- Speed Boat
- Vessel-ship
- Other

Image preprocessing and annotation conversion were carried out using the Hugging Face DetrImageProcessor. This processor automatically performs image resizing, normalization, tensor conversion, and bounding box formatting required by the DETR. This processor ensured that the images and annotations were in a consistent format when being supplied to the model for training and evaluation.

Training was performed using the AdamW optimiser, which decouples weight decay from the gradient-based update and is commonly used to improve generalisation in deep learning models (Loshchilov and Hutter 2019). A weight decay value of 0.0001 was used for all experiments.

6 Training and Fine-Tuning Process

6.1 Training Environment

Model development and training were conducted using Python, PyTorch, and the Hugging Face Transformers library. The training was carried out in a GPU-enabled environment that had four NVIDIA RTX 5000 graphics processing units, each with 32GB of VRAM. Using a batch size of 2, each training epoch needed 15-20 minutes training time depending on the experiment configuration.

The cleaned dataset was loaded using custom PyTorch Dataset and DataLoader classes. The same training, validation, and testing datasets were used for all experiments to ensure fair comparison between configurations.

6.2 Fine-Tuning Procedure

Transfer learning was applied by fine-tuning the pretrained DETR ResNet-50 model on the Singapore Maritime Dataset. During training, images and their corresponding COCO annotations were prepared using the DetrImageProcessor before being supplied to the model.

For every training iteration, image batches went through the network to generate object classification and bounding box predictions. The DETR loss function, which combines classification and bounding box regression losses, was then calculated automatically by the model. Gradients were computed using backpropagation, and model parameters were updated using the AdamW optimiser.

After each training epoch, the model was evaluated using the validation dataset. Validation loss was tracked during training to evaluate model convergence and identify the highest performing model. A checkpoint of the model was saved after every epoch. At the end of training, the model with the lowest validation loss was stored separately as the best model for that experiment.

6.3 Experimental Configurations

To investigate the effect of training duration and learning rate on model performance, three fine-tuning experiments were carried out. All the experiments used the same training, validation, and testing datasets. The learning rate and number of training epochs were varied.

Table 6.1 Summarises the configurations used for each experiment.

Experiment	**Learning Rate**	**Weight Decay**	**Epochs**
Experiment 1	1×10^{-5}	0.0001	5
Experiment 2	1×10^{-5}	0.0001	10
Experiment 3	5×10^{-5}	0.0001	5

Table 6.1: Experiment configurations.

Experiment 1 was designed to create a baseline fine-tuning configuration. It used a conserva-

tive learning rate and limited training duration. Experiment 2 adds more epochs to determine whether additional epochs would improve convergence and detection performance. Finally, experiment 3 increased the learning rate by a factor of five. This let us investigate whether faster optimisation would improve training efficiency or impact model stability.

To ensure a fair comparison, all experiments used the same DETR architecture, dataset splits, preprocessing pipeline, optimizer, batch size, and weight decay settings. Only the learning rate and number of epochs were altered between experiments.

The performance of each configuration was compared using validation losses and object detection metrics on unseen test data. The best-performing model was then selected for final evaluation and analysis.

7 Evaluation Metrics and Results

7.1 Evaluation Metrics

The performance of the fine-tuned DETR models was evaluated using both quantitative object detection metrics and qualitative visual inspection of results. Validation loss was monitored throughout training to assess model convergence and aid model selection.

For the final evaluation, Mean Average Precision (mAP), Mean Average Precision at an Intersection over Union Threshold of 0.50 (mAP@50), Mean Average Precision at an intersection over Union Threshold of 0.75 (mAP@75), and Mean Recall were calculated using the unseen testing dataset. These metrics are generally used for object detection tasks because they can evaluate both classification accuracy and bounding box performance.

The use of multiple evaluation metrics allows for a more complete assessment of model performance, as object detection is dependent on object classification in addition to precise localisation.

7.2 Baseline DETR Performance

Before fine-tuning the pretrained DETR ResNet-50 model was evaluated using its original COCO-trained weights. The purpose of this was to establish a baseline and assess how well the generic pretrained model could detect maritime objects without specific training.

As shown in Figure 7.1, the baseline model was able to detect several visible vessels in the scene, but it classified them using the generic COCO label “Vessel-ship”. While the detections show that the pretrained model has some understanding of maritime objects, it was not able to separate the dataset-specific classes such as Sail boat, Ferry, Buoy, or other. Additionally, the model is reporting one ship as multiple ships, and picking up smaller objects which are non-existent. This limitation was expected because the original DETR model was training on a broad variety of categories rather than the custom labels used in this project. The baseline results show the need for transfer learning, where the model is fine-tuned on specific annotations to improve accuracy and make the detections more relevant to the selected dataset.

Figure 7.1: Plotted Baseline DETR predictions.

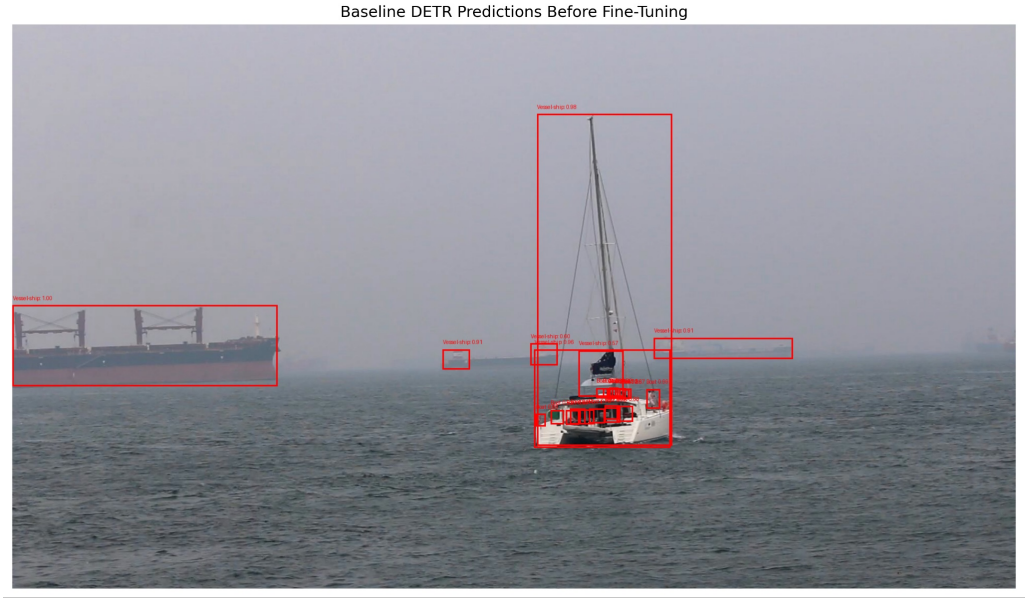

Figure 7.1: Plotted baseline DETR prediction.

7.3 Training Performance

Training and validation loss was monitored during the fine-tuning process, this was done to evaluate model convergence and pick the most effective training configuration.

For Experiment 1, both training and validation loss decreased consistently throughout all 5 epochs, indicating successful learning and convergence. The training loss decreased from approximately 1.95 during the first epoch to 1.02 by the last epoch, while validation loss decreased from approximately 1.55 to 0.98, this shows the model was successful at learning features without significant overfitting.

Figure 7.2: Training and validation loss curves for Experiment 1.

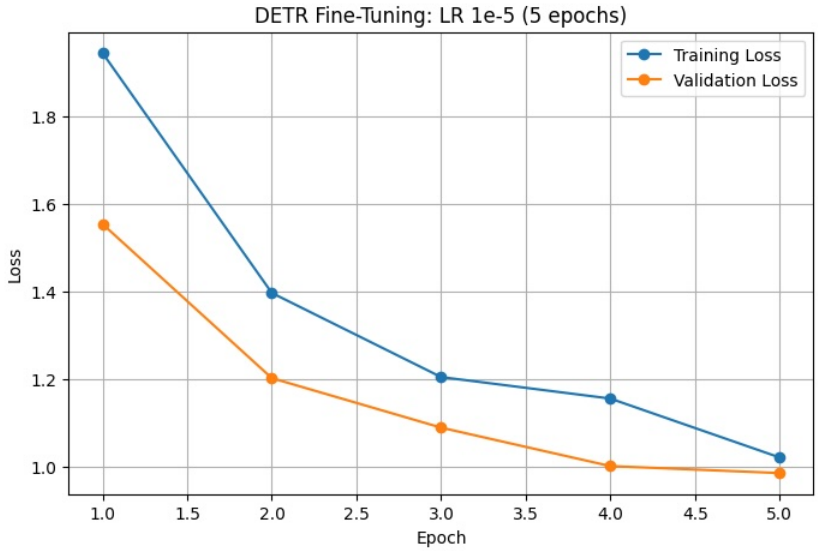

Figure 7.2: Training and loss curves for Experiment 1.

Experiment 2 increased the training period to ten epochs while keeping the same learning rate. Both training and validation losses decreased throughout training, with training loss reducing from approximately 1.87 to 0.71 and validation loss decreasing from 1.42 to 0.62. The lowest validation loss occurred at Epoch 10 (0.6185). This suggests that the model was reaching convergence, meaning additional training beyond ten epochs may provide limited gains while increasing the chance of overfitting.

Figure 7.3: Training and validation loss curves for Experiment 2.

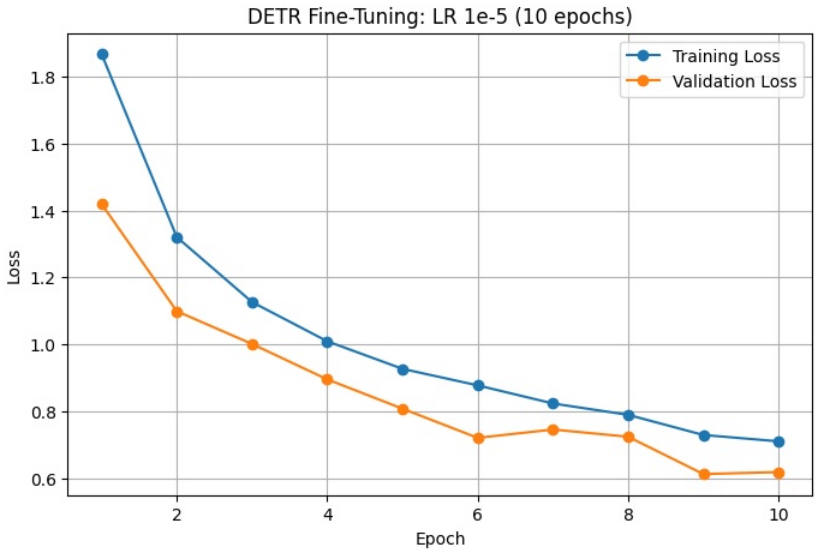

Figure 7.3: Training and loss curves for Experiment 2.

Experiment 3 looked at the impact of increasing learning rate from 1×10^{-5} to 5×10^{-5} while using the same training duration as experiment 1 (5 epochs). Although both training and validation losses eventually decreased throughout training, the optimisation process was significantly less stable than Experiments 1 and 2. The training loss erratically increased from 1.84 to 2.84 between Epochs 1 and 2. Then, it began gradually decreasing to 2.09 by the final epoch. Similarly, the validation loss decreased from 3.31 to 1.97 but was still substantially higher than the losses shown by the first two experiments. This indicates that the higher learning rate resulted in unstable optimisation. In turn, it prevented the model from converging as efficiently as the lower learning rate configurations.

Figure 7.4: Training and validation loss curves for Experiment 3.

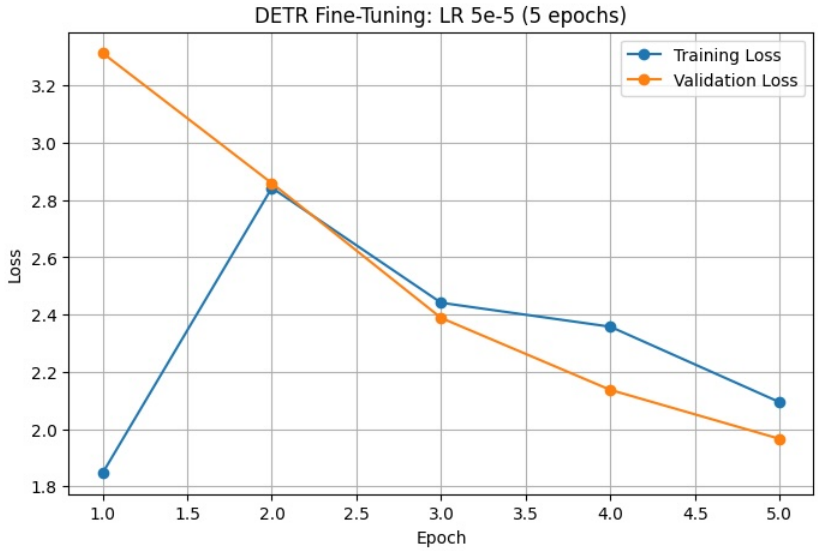

Figure 7.4: Training and loss curves for Experiment 3.

7.4 Experiment Comparison

To compare the effectiveness of each training configuration, validation losses from all three experiments were plotted together.

Figure 7.5: Validation loss curves for Experiments 1, 2, and 3.

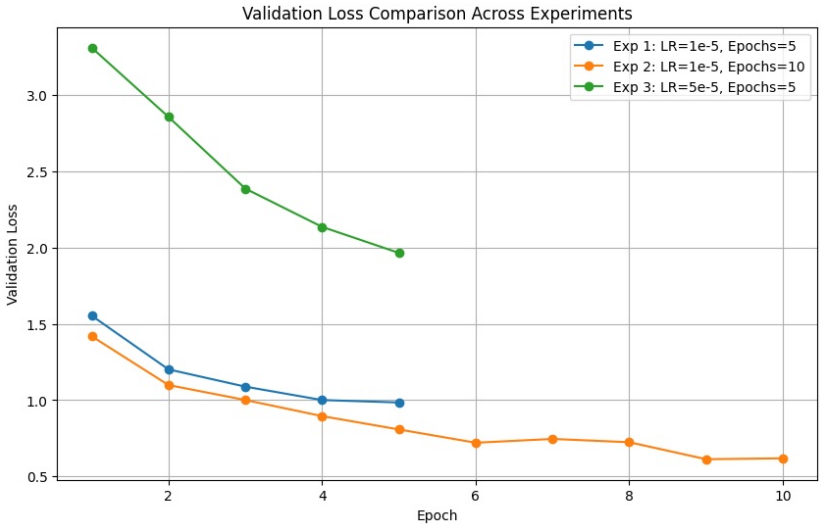

Figure 7.5: Validation loss curves for Experiments 1, 2, and 3.

The comparison demonstrates that experiment 2 achieved the lowest validation loss through training and therefore produced the best overall performance. Experiment 1 achieved good convergence but was undertrained in comparison to 2. Experiment 3 had the highest validation loss, showing that the larger learning rate led to less stable optimisation and worse generalisation.

Table 7.1 Summarises the final training and validation losses achieved by each experiment.

Experiment	Final Training Loss	Final Validation Loss	Best Validation Loss
Experiment 1	1.0206	0.9849	0.9849
Experiment 2	0.7103	0.6185	0.6126
Experiment 3	2.0941	1.9662	1.9662

Table 7.1: Summary of the final training and validation loss figures.

Based on the validation loss result, Experiment 2 was selected as the best model and was used for final testing and evaluation.

7.5 Final Model Evaluation

The unseen testing data was then used to evaluate the best model from each experiment. Mean Average Precision (mAP), mAP@50, mAP@75, and Mean recall were calculated to assess object detection performance.

Table 7.2 Summarises the final object detection metrics for each experiment on the test set.

Experiment	mAP	mAP@50	mAP@75	Mean Recall
Experiment 1	0.2181	0.3558	0.2131	0.3451
Experiment 2	0.3491	0.5443	0.3601	0.4634
Experiment 3	0.0285	0.0713	0.0185	0.0108

Table 7.2: Object detection evaluation metrics for each experiment on the test set.

The results show that experiment 2 outperformed the other configurations in all the evaluation metrics. The model achieved an mAP of approximately 0.349 and an mAP@50 of approximately 0.544. This shows the model was able to correctly identify and locate objects with good accuracy.

In comparison, Experiments 1 and 3 produced lower mAP scores. Experiment 1 was limited by insufficient training time, meaning it did not converge. On the other hand, Experiment 3 was negatively impacted by the instability caused from the increased learning rate. These findings demonstrate that extending training duration while preserving a conservative learning rate is the most effective fine-tuning strategy for this dataset.

7.6 Qualitative Detection Results

In addition to the quantitative evaluation metrics, qualitative inspection of the outputs was performed using unseen test images. Visual inspection gives further insight into the model's performance and can help identify strengths and weaknesses that might not be portrayed by numerical metrics.

To ensure fair comparison, the same test images were evaluated using the baseline pretrained

DETR model and the fine-tuned models from each experiment. This allowed the impact of the fine-tuning to be compared directly across identical scenes.

Figure 7.6: Qualitative comparison of baseline DETR and fine-tuned experiments on Test Image 1.

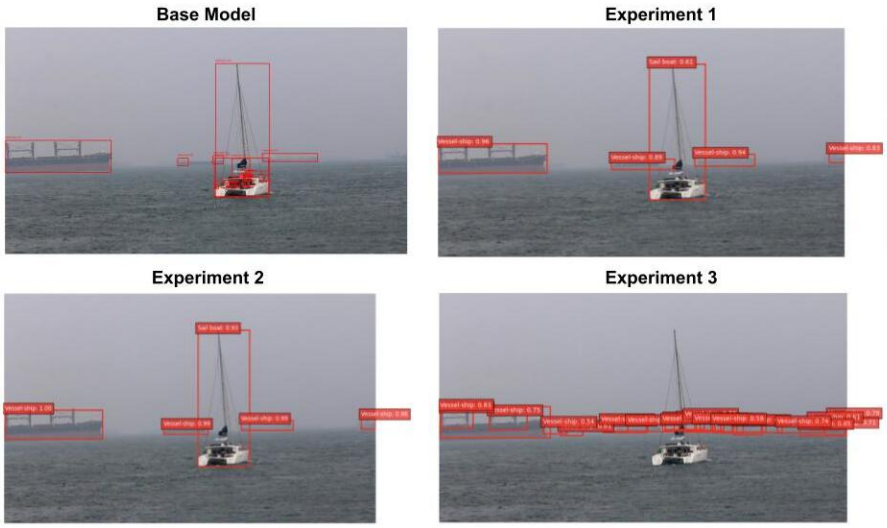

Figure 7.6: Prediction outputs of all experiments on image 1

Figure 7.7: Qualitative comparison of baseline DETR and fine-tuned experiments on Test Image 2.

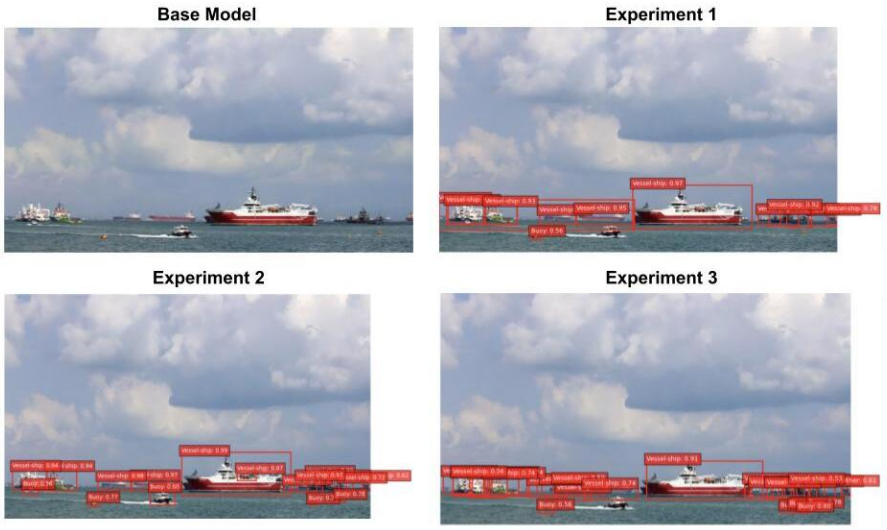

Figure 7.7: Prediction outputs of all experiments on image 2

The qualitative results show clear improvement in maritime object detection after fine-tuning. The baseline pretrained DETR model was able to detect some large objects but did not have knowledge of specific categories contained within the dataset. As a result, the detection was less informative and object localisation was less precise. This can clearly be seen in image 2, where no detections were made.

Experiments 1 and 2 displayed improvements over the baseline model. Both configurations

were able to identify objects such as Vessel-ship, Sail Boat, and Buoy, while producing accurate bounding boxes around the detected objects. Experiment 2 was slightly better, correctly identifying more objects with a higher confidence across both test images. The model was able to detect multiple objects in a busy scene while maintaining accurate localisation and minimal redundant detections.

Experiment 3 on the other hand, did not perform as well. Although vessels were successfully detected, the model generated many redundant, overlapping bounding boxes, partially around distant objects that were not as clear. This behaviour aligns with the higher validation losses and shows that the increased learning rate significantly impacted model performance. The excessive number of detections indicates that the model struggled to converge at an optimal solution when compared with Experiments 1 and 2.

Some limitations remained across all models. Smaller objects, distant vessels, and less commonly represented classes were not detected reliably. These difficulties are likely caused by the class imbalance present within the dataset, where there were significantly more Vessel-ship instances.

Overall, the qualitative results strongly correlate with the quantitative metrics. Experiment 2 produced the most reliable detections, cleanest detection, and strongest overall performance, reinforcing its selection as the final model for maritime object detection.

8 Analysis of Performance and Potential Biases

Experiment 2 showed the strongest results, with the best mAP and recall numbers being the highest. The lower learning rate combined with the extra epochs allowed the model to converge better than the other configurations. Experiment 3 on the other hand showed that increasing learning rate severely impacted optimisation stability, this led to poor convergence and weak performance.

The Singapore Maritime Dataset's biggest limitation was the class imbalance. The majority of its labels were a part of the Vessel-ship category, meaning the model had substantially more examples to learn in this class. This meant large vessels were detected more reliably than underrepresented classes like Buoy, Kayak, Sail boat, and Flying bird-plane.

Maritime images are highly complex, which creates challenges. Many objects were small, far away, partially occluded, or close to the horizon. Therefore, accurate detection was difficult and likely contributed to missed detection and localisation errors. This can be seen when inspecting the mAP scores, mAP@75 was lower than mAP@50, hinting that the predicted bounding boxes were not precise enough for stricter localisation requirements.

Despite this, fine-tuning DETR still improves maritime object detection performance. But, class imbalance, small object detection, and localisation accuracy can still be improved in future iterations. Potential improvements include data augmentation, class balancing techniques, and additional training with early stopping.

9 Ethical and Societal Implications

Maritime object detection systems can improve safety and situational awareness, but they must be used responsibly. False negatives, where vessels or objects are missed, could create safety risks in autonomous navigation. False positives could also lead to poor operational decisions or unnecessary warnings. Thus, this type of system should be used to support human decision-making, and not replace humans entirely.

Dataset bias is another concern. Since the dataset is heavily weighted towards the Vessel-ship category, the model may perform better on common vessel types. This means rarer classes like Boat, Kayak, Sail boat, or Flying bird-plane may not be detected. This could reduce reliability in the environment, such as a recreational lake, where these smaller or rarer objects are important.

Privacy and responsible data use must also be considered because maritime surveillance systems collect vast amounts of visual data from vessels, ports, and coastal areas. This data may be sensitive and must be secured and processed according to guidelines. While the system has potential for real-world use, it requires further human testing, human oversight, and a more balanced dataset before being deployed in safety-critical environments.

10 Conclusion

This project successfully developed and evaluated a maritime object detection system using the DETR architecture and transfer learning. A pretrained DETR ResNet-50 model was fine-tuned on the Singapore Maritime Dataset, which had 5446 images and 41,747 object annotations across multiple maritime categories.

Three fine-tuning configurations were tested to analyse the impact of training duration and learning rate on performance. The results demonstrated that Experiment 2, which used a learning rate of 1×10^{-5} and 10 training epochs, had the strongest performance. A significant improvement when compared with the baseline model and the other training configurations could be seen in both the quantitative and qualitative evaluation results.

Furthermore, the project brought to light key challenges with maritime object detection. This includes class imbalance, small object detection, and localisation accuracy in complex scenes. Despite these limitations, the fine-tuned DETR model was able to identify and localise maritime objects across a broad range of unseen images.

Overall, the results demonstrated that transformer-based object detection models can be adapted to maritime environments through transfer learning and fine-tuning, providing a good foundation for applications such as maritime surveillance, autonomous vessels, and port monitoring.

REFERENCES

Carion, Nicolas et al. (2020). "End-to-End Object Detection with Transformers". In: arXiv: 2005.12872 [cs.CV]. URL: https://doi.org/10.48550/arXiv.2005.12872.

Lin, Tsung-Yi et al. (2015). *Microsoft COCO: Common Objects in Context*. arXiv: 1405.0312 [cs.CV]. URL: https://arxiv.org/abs/1405.0312.

Loshchilov, Ilya and Frank Hutter (2019). "Decoupled Weight Decay Regularization". In: URL: https://openreview.net/forum?id=Bkg6RiCqY7.

Maritime (Mar. 2023). *Singapore Maritime Dataset*. https://universe.roboflow.com/maritime-cumkb/singapore-maritime. Open Source Dataset. URL: https://universe.roboflow.com/maritime-cumkb/singapore-maritime.